Lập trình mạng · Chương 4

RPC: truyền tham số, idempotency và truy cập đồng thời trong RMI

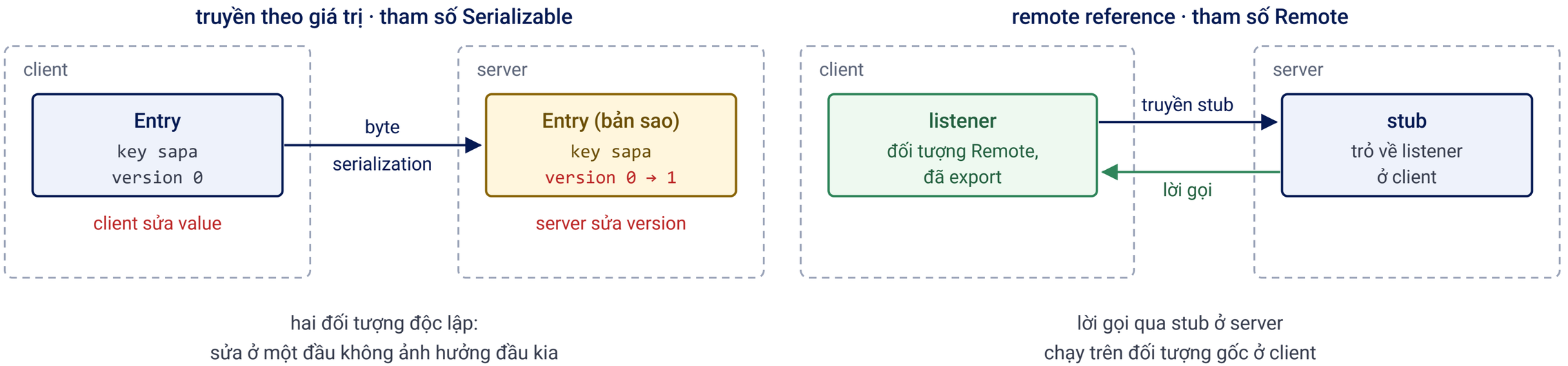
TS. Đặng Ngọc Hùng

Thêm thao tác cập nhật: ba câu hỏi mới cho lời gọi từ xa

1. Tham số `Entry` đi qua mạng: server nhận **chính đối tượng** của client hay một **bản sao**?
2. Gọi lại sau RemoteException: cập nhật có bị **áp dụng hai lần**?
3. Nhiều client cùng cập nhật một khóa: remote object có còn **đúng** khi các lời gọi chạy song song?

> **Câu hỏi của phần:** RMI truyền tham số thế nào, gọi lại sao cho an toàn, và remote object chịu truy cập đồng thời ra sao?

Tham số Serializable thành bản sao, tham số Remote thành stub



hai đối tượng độc lập:
sửa ở một đầu không ảnh hưởng đầu kia

lời gọi qua stub ở server
chạy trên đối tượng gốc ở client

Truyền theo giá trị: tham số Serializable thành bản sao độc lập. **Remote reference**: tham số Remote thành stub trỏ về đối tượng gốc.

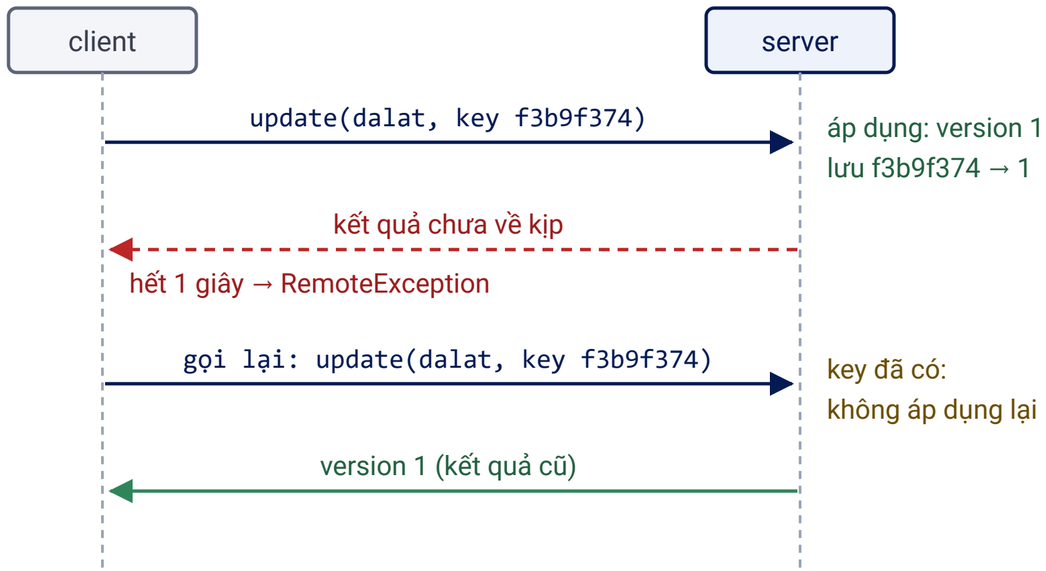
Idempotency key cho phép gọi lại mà không áp dụng hai lần

gọi hai lần thay vì một lần

`lookup("sapa")`
dữ liệu không đổi → idempotent

`đặt value = "X"`
vẫn là value = "X" → idempotent

`update: version + 1`
version tăng 2 → không idempotent



Idempotency key: một key cho mỗi thao tác, dùng lại khi gọi lại; server lưu key đã xử lý và trả kết quả cũ.

Sửa bản sao ở một đầu, đầu kia không thấy

Java

```
Entry entry = new Entry("sapa", "Thị trấn vùng cao");
long version = service.update(entry, null);  // entry đi qua serialization: server nhận một bản sao
System.out.println("server returned version " + version + "; client entry.version = " + entry.version);
entry.value = "sửa ở client";  // chỉ sửa bản của client
System.out.println("server entry: " + service.lookup("sapa"));
```

Kết quả chạy

```
server returned version 1; client entry.version = 0
client entry: sapa = sửa ở client (version 0)
server entry: sapa = Thị trấn vùng cao (version 1)
```

Không có key, cập nhật bị áp dụng hai lần; có key, chỉ một lần

Java

```
long version = requestId == null
        ? apply(entry)  // không có idempotency key: mỗi lời gọi là một lần cập nhật
        : processed.computeIfAbsent(requestId, id -> apply(entry));  // key đã xử lý: trả kết quả cũ
if (pause && paused.add(entry.key)) {
    sleep(1_500);  // mô phỏng server chậm: đã cập nhật nhưng chưa trả kết quả
}
```

Kết quả chạy

```
no idempotency key:
  attempt 1: UnmarshalException, cause java.net.SocketTimeoutException: Read timed out
  attempt 2: version 2
idempotency key f3b9f374:
  attempt 1: UnmarshalException, cause java.net.SocketTimeoutException: Read timed out
  attempt 2: version 1
```

RMI có thể chạy các lời gọi đồng thời: remote object phải an toàn

Java

```
Entry old = unsafeData.get(entry.key);  // đọc rồi ghi: hai thread có thể đọc cùng một version
unsafeData.put(entry.key, new Entry(entry.key, entry.value, old == null ? 1 : old.version + 1));

Entry stored = data.compute(entry.key,  // đọc và ghi nguyên tử cho từng khóa
        (k, old) -> new Entry(k, entry.value, old == null ? 1 : old.version + 1));
```

Kết quả chạy

```
UnsafeEntryService: 2000 updates -> version 1938
EntryService: 2000 updates -> version 2000
```

- RMI **không quy định** lời gọi chạy trên thread nào: hai lời gọi đến cùng lúc chạy trên hai thread
- HashMap đọc rồi ghi: mất 62 lần cập nhật; **ConcurrentHashMap**.compute: đọc và ghi nguyên tử cho từng khóa

Câu hỏi 1

Phương thức RMI ở server đặt entry.version = 1 trên tham số Entry (Serializable) mà nó nhận được. Sau lời gọi, Entry của client ra sao?

A. version thành 1, vì RMI truyền tham chiếu tới đối tượng của client.

B. version vẫn như cũ, vì server sửa trên bản sao của nó.

C. version thành 1 ở lần client gọi lookup tiếp theo.

D. Lời gọi ném RemoteException vì tham số bị sửa.

Câu hỏi 1

Phương thức RMI ở server đặt entry.version = 1 trên tham số Entry (Serializable) mà nó nhận được. Sau lời gọi, Entry của client ra sao?

A. version thành 1, vì RMI truyền tham chiếu tới đối tượng của client.

B. version vẫn như cũ, vì server sửa trên bản sao của nó.

C. version thành 1 ở lần client gọi lookup tiếp theo.

D. Lời gọi ném RemoteException vì tham số bị sửa.

Tham số Serializable được truyền theo giá trị: server deserialization thành bản sao riêng. Muốn client biết kết quả thì server trả qua giá trị trả về.

Câu hỏi 2

Client gọi phương thức update (ghi value, tăng version), nhận RemoteException rồi gọi lại. Vì sao lần gọi lại phải mang cùng idempotency key với lần đầu?

A. Để server xác thực client trước khi cập nhật.

B. Để RMI dùng lại kết nối TCP của lần đầu.

C. Vì không có key thì lần gọi lại chắc chắn thất bại.

D. Vì lần đầu có thể đã được áp dụng; cùng key giúp server nhận ra lần gọi lặp và trả kết quả cũ.

Câu hỏi 2

Client gọi phương thức update (ghi value, tăng version), nhận RemoteException rồi gọi lại. Vì sao lần gọi lại phải mang cùng idempotency key với lần đầu?

A. Để server xác thực client trước khi cập nhật.

B. Để RMI dùng lại kết nối TCP của lần đầu.

C. Vì không có key thì lần gọi lại chắc chắn thất bại.

D. Vì lần đầu có thể đã được áp dụng; cùng key giúp server nhận ra lần gọi lặp và trả kết quả cũ.

RemoteException không cho biết server đã thực hiện hay chưa. Lần gọi lại mang cùng key để server nhận ra thao tác đã áp dụng và không áp dụng lần nữa; key mới sẽ bị coi là thao tác mới.

Tổng kết

1. Tham số Serializable: **truyền theo giá trị**, hai bản độc lập; tham số Remote: **remote reference**, server nhận stub

2. Update không có **idempotency**: gọi lại cần **idempotency key**; server lưu key và trả kết quả cũ

3. RMI có thể chạy các lời gọi đồng thời: remote object dùng **ConcurrentHashMap**, đọc và ghi nguyên tử

Từ khóa cần nhớ truyền theo giá trị | remote reference | idempotency | idempotency key | ConcurrentHashMap

Mở rộng

Chủ đề nâng cao

- Distributed garbage collection của RMI
- Callback từ server về client bằng remote reference

Bài tập mở rộng

Viết test gọi update 100 lần đồng thời từ 10 client với cùng bộ idempotency key; kiểm tra mỗi key chỉ được áp dụng một lần.

Tiếp theo

Web service: SOAP, hợp đồng WSDL và message XML